DAY 2 · MODULE 5 · 25 MIN

Foundry Toolbox in Practice

Managed tools you attach without writing

Where we are in Actions

Module 4 covered the function-calling contract — same for every tool.

- Module 5 (this one) — Toolbox tools — Microsoft-managed catalog of tools you attach without writing
- Module 6 — Custom function tools you author in MAF

Toolbox is where you say: 'I need my agent to search the web. I don't need to build a web-search tool.'

What Foundry Toolbox is

- A curated catalog of ready-to-use tools an agent can attach to
- A project-scoped resource — one toolbox per set of tools you want to reuse
- Exposed to MAF over an MCP endpoint — same MCP protocol as Day 3
- Versioned — publish v1, iterate to v2, consumers pin (or use the default)
- Managed by Microsoft (or your organization) — you don't own the runtime

One toolbox can contain many tools. Multiple agents can consume the same toolbox.

The tools available in the catalog

Category	Examples
Web	Web search (Bing), Bing Custom Search
Code	Code interpreter (sandboxed Python)
Data & search	Azure AI Search, File search
Enterprise	SharePoint, Microsoft Fabric, WorkIQ
Custom	Any MCP server (local or remote), Skills
Discovery	Tool search (intent-based routing)

Two worth extra attention: MCP tools (attach any MCP server) and Toolbox search (LLM-driven routing).

Why toolbox, and not just tools=[...] on the agent

Function tools attach to a specific agent. Toolboxes are reusable across agents.

Question	Function tools	Toolbox
Where is the code?	In your app	In Foundry / MCP server
Who runs it?	Your process	Foundry or the MCP server
Auth to internal systems?	Your code	Project connections + MI
Shared across agents?	Copy code	One toolbox, many agents
Versioned?	With your app	First-class in Foundry
Governance?	You audit	Central catalog, RBAC, rai_config

Toolbox wins on reuse, governance, and platform auth. Function tools win on flexibility.

D E M O

~5 min

Consume a hosted toolbox from your agent

Client-side MAF agent hitting a pre-published Foundry toolbox over MCP. Show the toolbox exists (portal or CLI), walk the ~15-line consumer pattern — DefaultAzureCredential + get_bearer_token_provider fed into MCPStreamableHTTPTool(header_provider=…), then tools=[toolbox_tool] on the agent. Run once with a question that exercises a tool. Point at the trace: MAF calls the tool exactly like a local function tool.

Runbook: demos/day2/module-5-demo-1-attach-toolbox.md · Sample: demos/day2/module-5-demo-1-attach-toolbox/

Toolbox anatomy

A toolbox is a project resource with:

- Name — identifier
- Description — human-readable
- Tools — a list of tool entries (each with its own config)
- Connections — project connections the toolbox references (AI Search, MCP server, Bing…)
- Skills — Foundry skills packaged as MCP resources
- Policies — optional RAI policy applied at the toolbox level
- Versions — the whole thing is versioned; one version is the default

Same 'publish version, consumers pin' pattern as Prompt agents from Day 1.

Two authoring paths

- Portal
  - Click-through UI in the Foundry portal for exploration
- IaC-first (workshop path)
  - azd ai CLI or SDK
  - Two-step flow:
    - 1. Create the connections (one per credential record)
    - 2. Create the toolbox from a YAML file

YAML references connections by name. Credentials never live in YAML — they live in connections.

A minimal toolbox YAML

```
# my-toolbox.yaml
description: Docs assistant helper toolbox

tools:
  - type: web_search
    name: web

  - type: code_interpreter
    container: { type: auto }
    name: code

  - type: toolbox_search   # intent-based routing over the tools above
```

Create it:

```
azd ai project set $PROJECT_ENDPOINT
azd ai toolbox create my-toolbox --from-file ./my-toolbox.yaml
```

Adding a tool that needs a connection

For tools that reach external systems, register a connection first:

```
azd ai connection create my-search-conn \
  --kind cognitive-search \
  --target https://<search>.search.windows.net \
  --auth-type project-managed-identity
```

Then reference by name in the toolbox YAML:

```
tools:
  - type: azure_ai_search
    name: search
    azure_ai_search:
      indexes:
        - project_connection_id: my-search-conn
          index_name: docs-index
```

Managed identity means the toolbox authenticates as itself — no long-lived keys.

Attaching an MCP server as a Toolbox tool

Any MCP server (public URL or your own) can be a toolbox tool:

```
tools:
  - type: mcp
    server_label: myserver
    server_url: https://your-mcp-server.example.com
    require_approval: never
    project_connection_id: my-mcp-conn
```

- Auth flows through the connection — keys, OAuth, Entra tokens all supported
- Same MCP protocol as Day 3 — Toolbox is one way to wire an MCP server in

Day 3 wires the Azure DevOps MCP server directly (not through Toolbox) — same protocol, different attachment.

Consuming a toolbox — the two endpoints

A toolbox exposes two MCP endpoints:

Role	Endpoint	Use
Consumer	{project}/toolboxes/{name}/mcp?api-version=v1	Always default version. Agents connect here.
Developer	{project}/toolboxes/{name}/versions/{v}/mcp?api-version=v1	Version-pinned. For testing before promoting.

Rule of thumb: agents use the consumer endpoint. Promote new versions without redeploying the agent.

Attaching a toolbox to an agent (MAF, Python)


```
from agent_framework import Agent
from agent_framework.foundry import FoundryChatClient
from agent_framework.foundry.toolbox import FoundryToolbox
from azure.identity import AzureCliCredential

toolbox = FoundryToolbox(
    project_endpoint="https://<foundry-resource>.services.ai.azure.com/api/projects/<your-project>",
    name="my-toolbox",
    credential=AzureCliCredential(),
)

agent = Agent(
    client=FoundryChatClient(credential=AzureCliCredential()),
    name="DocsAssistant",
    instructions="Use the toolbox tools to answer questions and cite sources.",
    tools=[toolbox],   # ← one line to attach all toolbox tools
)
```

toolbox behaves as a tool collection. All its tools become available via the Module 4 function-calling contract.

Attaching a toolbox to a Prompt agent

For Prompt agents, the toolbox is attached in the agent configuration, not from code:

- Portal: Agents → your agent → Tools → Add Toolbox → select → save
- SDK: pass toolbox reference in PromptAgentDefinition(..., toolbox=...)
- YAML: reference the toolbox in the agent's declarative definition

Consuming agent code (FoundryAgent(agent_name=..., agent_version=...)) stays the same. All Prompt-agent version consumers automatically get the new tools.

Same pattern for Hosted agents — configuration, not code.

Governance you get from toolbox

Because toolboxes are a first-class Foundry resource:

- RBAC — who can create, update, and consume a toolbox is Entra-controlled
- Audit — toolbox versions and consumption tracked centrally
- RAI policy — apply a Responsible AI policy at the toolbox level (policies.rai_config)
- Central catalog — one place to see every managed tool in the project

The platform answer to: 'how do we stop developers from wiring random tools into production agents?'

Versioning workflow

Standard workflow you'll use:

- Create v1 → automatically the default → agents consume via the consumer endpoint
- Author v2 (add a tool, tighten a description, swap an MCP connection)
- Test against the developer endpoint (/versions/2/mcp) before promotion
- Promote v2 to default when validated → agents pick it up on their next call, no code change

The promote step is the 'ship' moment. Same discipline as Prompt agent versioning from Day 1.

When to build a function tool instead

Custom function tool (Module 6)

- Logic doesn't fit any Toolbox catalog entry
- Runtime context Toolbox model can't express (per-user filtering, session state)
- Prototyping fast — don't want to publish a toolbox yet
- Tool talks to something inside your app's process (in-memory cache, local model)

Foundry Toolbox

- Tool is a shared capability across agents
- Want central governance, RBAC, and audit
- Integrates with a supported enterprise system (SharePoint, Fabric, Bing, AI Search, MCP)
- Want versioning as a first-class concern

Mix both. Real agents often have tools=[my_function_tool, my_toolbox].

Common traps

- Credentials in the YAML — don't. Use connections. YAML is checked into source.
- Consumer vs. developer endpoint confusion — agents = consumer; test = developer.
- Not testing before promoting — v2 is default the moment you promote. Test /versions/2/mcp first.
- Too many tools in one toolbox — ≤10 per agent applies. Use toolbox_search when >10.
- Assuming toolbox tools are free — some have per-call costs (Bing, code interpreter). Budget.
- Skipping RAI policy — toolbox is the right place to enforce content safety centrally. Use it.

Takeaways

- Foundry Toolbox = a curated catalog of managed tools you attach without writing.
- Toolbox tools are exposed over MCP — same protocol as Day 3.
- Two-step authoring: connections first, then a YAML that references them by name.
- Two endpoints: consumer (default version) and developer (version-pinned).
- Toolbox wins on reuse, governance, platform-managed auth. Function tools win on flexibility.
- Mix both — real agents often have custom + toolbox tools together.

Next: Authoring your own function tools in MAF — hands-on.